

ICMP Blind Attacks against TCP

Blind Connection Reset and Blind Throughput Reduction

Shemanty Mahjabin, student ID: 2105091

Mahabuba Sharmin Mim ,student ID:2105103

CSE 406: Computer Security Sessional
CSE,BUET

Design Report

Project Overview and Ethical Scope

This project demonstrates how crafted ICMP error messages can influence a TCP transfer when an endpoint trusts unauthenticated network feedback. Two assigned effects are studied:

- termination after a forged ICMP hard error; and
- reduced sending rate after a forged ICMP Source Quench.

The graded demonstration uses a **group-written user-space TCP experiment harness** with selectable legacy and hardened ICMP-handling modes. A current Linux TCP stack may also be observed as a standards-compliant comparison on the development platform used for the final demonstration.

All traffic remains inside an owned, isolated Linux-based namespace topology. We will develop on native Linux or WSL2 while preserving the same logical network design. No public, campus, or third-party host is targeted.

Source: RFC 5927; RFC 6633

Definition of the Two Attacks

1. Blind connection-reset attack

A forged ICMP Destination Unreachable hard error quotes identifiers of an existing TCP flow. A legacy implementation may treat the report as fatal and abort the connection.

2. Blind throughput-reduction attack

A forged ICMP Source Quench message quotes the same flow. Historical behavior could push the sender toward slow start or a lower sending rate.

3. Why “blind”?

The attacker is outside the Client-to-Server forwarding path and does not observe live packets. The laboratory four-tuple is deliberately fixed and supplied before the trial.

Historical Behavior and Current Reality

Question	Historical or legacy behavior	Current defended behavior
ICMP hard error	May abort the associated TCP connection.	Treat as advisory in a synchronized state; do not immediately close.
ICMP Source Quench	May reduce congestion window or sending rate.	Must be silently discarded for transport control.
Validation	Four-tuple matching may be sufficient.	Validate syntax, checksum, quoted tuple and, where available, sequence range.

Design consequence: harmful behavior is reproduced only in an explicitly labelled compatibility mode. Failure to affect a current Linux kernel is an expected defense result, not a failed project.

Source: RFC 1122; RFC 5927; RFC 6633

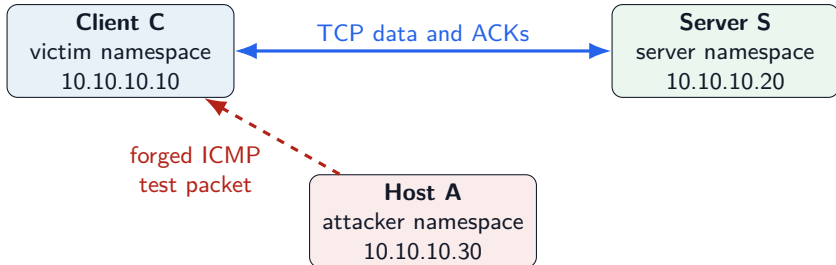
Project Objectives

1. Build an original IPv4 and ICMP packet-construction module using Python socket, struct, and checksum logic.
2. Demonstrate connection termination and throughput reduction in a controlled vulnerable mode.
3. Repeat identical packets against a hardened mode and compare connection state and goodput.
4. Verify packet fields and checksums through tcpdump or Wireshark.
5. Explain why current operating systems normally resist the historical Source Quench attack.



Network Topology and Components

Isolated Linux test environment — internal virtual bridge



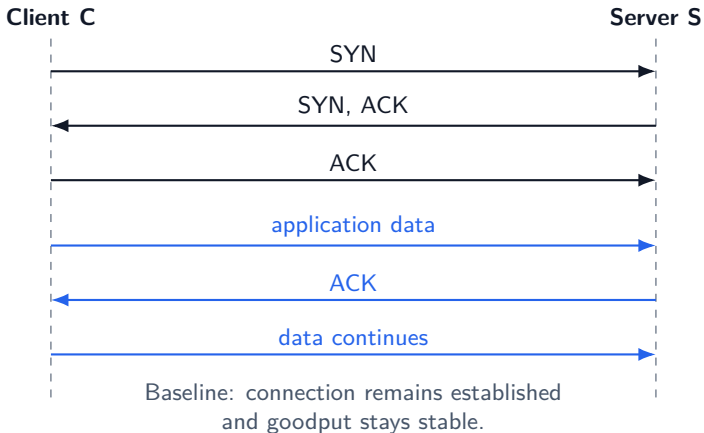
- Host A is not a router and does not capture the live Client–Server flow.
- The namespaces have no default route outside the isolated bridge.
- The same logical topology can run on native Linux or inside WSL2.
- tcpdump or Wireshark is used only for verification.

Network Components and Roles

Component	Role
Client C	Sends a fixed TCP workload, receives ICMP test traffic, and applies either legacy or hardened policy.
Server S	Receives the TCP workload; records delivered bytes, completion time, and premature closure.
Host A	Builds and transmits the selected ICMP error using the preconfigured laboratory flow identifiers.
Linux bridge	Connects three veth pairs on 10.10.10.0/24 without an external route.
Packet observer	Captures ICMP packets and endpoint logs for evidence. It does not provide flow information to Host A during a blind trial.

Required software: a Linux-compatible environment (native Linux or WSL2), Python 3, iproute2, tcpdump and optional Wireshark.

Normal TCP Timing



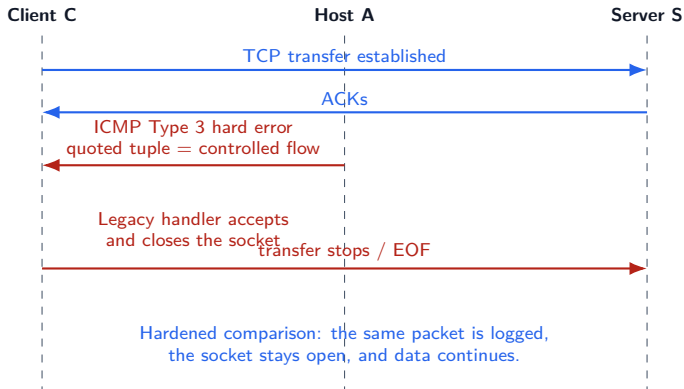
Attack Strategy A: Blind Connection Reset

1. Establish a fixed Client-to-Server TCP transfer and record five seconds of baseline activity.
2. Host A builds ICMP Destination Unreachable, Type 3, using a selected hard-error code and an embedded quotation matching the controlled flow.
3. The packet is delivered to Client C without observing the live TCP exchange.
4. In **Legacy vulnerable mode**, the handler associates the quotation with the active flow and closes the socket.
5. In **Hardened mode**, the same report is logged as nonfatal and the synchronized TCP transfer continues.

Important: this is not a forged TCP RST segment. The reset is the endpoint's reaction to an ICMP hard-error report.

Source: RFC 5927, Section 5

Blind Connection Reset Timing Diagram



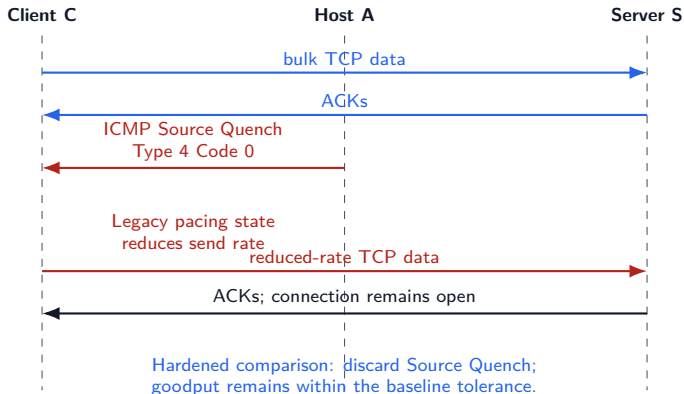
Attack Strategy B: Blind Throughput Reduction

1. Start the same fixed TCP workload and measure baseline goodput.
2. Host A sends ICMP Source Quench, Type 4 Code 0, quoting the controlled flow.
3. In **Legacy vulnerable mode**, each accepted message activates a predefined low-rate application pacing state.
4. The connection remains open, but fewer useful application bytes arrive per second.
5. In **Hardened mode**, Source Quench is discarded for transport control and goodput remains near baseline.

The low-rate compatibility state is an educational reproduction of historical behavior. It is not claimed to modify the current Linux congestion window.

Source: RFC 5927, Section 6; RFC 6633

Blind Throughput Reduction Timing Diagram



Blind Threat Model and Assumptions

Assumption	Meaning in this project
Known endpoint addresses	Private addresses are fixed by the setup script.
Known TCP ports	The client source port and server destination port are fixed for repeatability.
No live sniffing	Host A does not capture or adapt to Client–Server packets during the trial.
Controlled quotation	A laboratory sequence value is configured before the test; no brute-force search is used.
Owned environment	All three nodes exist inside one isolated Linux test environment; this may be native Linux or WSL2.

This proposal tests the ICMP trust and validation decision. It does not claim to reproduce the cost of discovering an unknown Internet connection.

ICMP Error Encapsulation

Outer IPv4 header — 20 bytes minimum
ICMP header — Type, Code, Checksum, 32-bit type-specific area
Quoted original IPv4 header — 20 bytes minimum
Quoted TCP prefix — source port, destination port, sequence number — 8 bytes

Minimum outer IPv4 packet in this design: $20 + 8 + 20 + 8 = 56$ bytes

The embedded quotation is what allows the receiver to associate the ICMP error with a candidate TCP connection.

Source: RFC 792; RFC 5927

Packet and Frame Structure

Layer or block	Length	Fields used
Ethernet header	14 bytes	Local delivery; normally supplied by the kernel when a raw IP socket is used.
Outer IPv4 header	20 bytes	Version, length, TTL, Protocol 1, source, victim destination and checksum.
ICMP header	8 bytes	Type, code, checksum and zero/type-specific word.
Quoted IPv4 header	20 bytes	Victim source, server destination and Protocol 6 TCP.
Quoted TCP prefix	8 bytes	Fixed source port, destination port and 32-bit sequence value.

Network byte order is big-endian. IPv4 and ICMP checksums are calculated after their checksum fields are set to zero.

Direction-Specific ICMP Fields

Field	Reset test	Throughput test
Outer destination	Client C, 10.10.10.10	Client C, 10.10.10.10
ICMP Type	3, Destination Unreachable	4, Source Quench
ICMP Code	selected hard-error code	0
Quoted IP source	Client C, 10.10.10.10	Client C, 10.10.10.10
Quoted IP destination	Server S, 10.10.10.20	Server S, 10.10.10.20
Quoted TCP ports	fixed client port → 5001	fixed client port → 5001
Quoted sequence	configured laboratory value	configured laboratory value
Expected legacy action	close matching transfer	enter low-rate pacing state

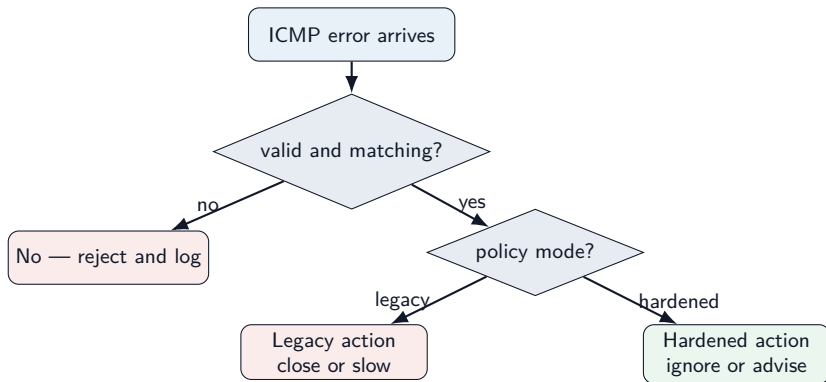
What Must Match the Controlled Flow

- Quoted IPv4 source address = Client C.
- Quoted IPv4 destination address = Server S.
- Quoted TCP source and destination ports = fixed laboratory ports.
- Quoted protocol = TCP, protocol number 6.
- Packet lengths and checksums are valid.
- In the hardened conceptual check, the quoted sequence lies in outstanding data:
 $SND.UNA \leq SEG.SEQ < SND.NXT$.



Source: RFC 5927, Section 4.1

Validation Decision

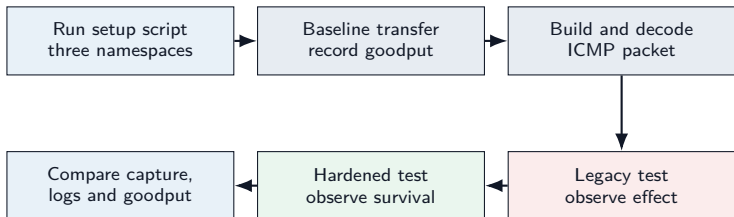


Selected Implementation Architecture

Module	Function
setup_lab.sh	Creates and removes the Linux bridge, namespaces, veth links and fixed private addresses.
tcp_sender.py	Sends the fixed workload; exposes flow identifiers; applies legacy or hardened ICMP policy; prints goodput.
tcp_receiver.py	Receives and verifies the workload; reports completion, bytes and premature closure.
icmp_builder.py	Serializes outer IPv4, ICMP and quoted IPv4/TCP fields and computes checksums.
run_trial.py	Runs baseline, attack and defense conditions with fixed timing and synchronized logs.

No ready-made attack library implements the graded packet mechanism. The packet analyzer is used only to verify the group's generated bytes.

Implementation Workflow



The same packet bytes are reused in the legacy and hardened comparisons so the policy is the only intended variable.

Packet Builder Algorithm

1. Refuse any destination outside the configured private lab subnet.
2. Serialize the quoted TCP source port, destination port and sequence in network byte order.
3. Build the quoted IPv4 header with Protocol 6 and calculate its checksum.
4. Prepend ICMP Type, Code, zero checksum field and the quoted datagram; calculate the ICMP checksum.
5. Build the outer IPv4 header with Protocol 1 and transmit through a raw socket.
6. Record type, code, addresses, ports, checksum, length and timestamp.

Verification: tcpdump or Wireshark must decode a valid 56-byte minimum IPv4 ICMP packet with the expected embedded TCP identifiers.

Experiment Sequence

Condition	Purpose	Expected evidence
Baseline	Confirm stable TCP transfer without ICMP intervention.	Complete payload and stable goodput.
Legacy reset	Demonstrate the historical fatal hard-error response.	Valid Type 3 packet, fatal decision, incomplete transfer.
Hardened reset	Test resistance to the identical hard error.	Packet logged; transfer continues.
Legacy throughput	Demonstrate historical Source Quench response.	Valid Type 4 packet and visible goodput reduction.
Hardened throughput	Test RFC 6633 behavior.	Packet discarded; goodput remains close to baseline.

One complete controlled trial will be performed for each condition. If implementation time permits, each condition will be repeated up to three times to check repeatability. The same payload size and timing configuration will be maintained across comparable trials.

Measurements and Success Criteria

Goodput

$$\text{Goodput} = \frac{\text{application payload bytes received} \times 8}{\text{measurement duration}}$$

Throughput reduction

$$\text{Reduction (\%)} = 100 \times \frac{\text{baseline goodput} - \text{attacked goodput}}{\text{baseline goodput}}$$

- Reset succeeds in legacy mode if the valid ICMP event aligns with premature transfer termination.
- Throughput reduction succeeds if legacy goodput visibly falls after Source Quench while the connection stays open.
- Defense succeeds if identical packets are logged but no unsafe state change occurs: the reset test connection survives, and the throughput test does not enter the predefined low-rate state.

Evidence Collected for Each Demonstration

Packet evidence

- ICMP Type and Code
- valid checksum
- quoted IP addresses
- quoted TCP ports
- event timestamp

Endpoint evidence

- policy decision
- connection state
- bytes delivered
- completion or early stop

Performance evidence

- baseline goodput
- legacy goodput
- hardened goodput
- percentage reduction

Packet capture + endpoint log + measured behavior must agree. A terminal message alone is not sufficient evidence.

Expected Outcome: Connection Reset

Legacy vulnerable mode

- Valid ICMP hard error reaches Client C.
- Quoted identifiers match the active flow.
- Handler records a fatal decision.
- Sender closes before full payload delivery.
- Server reports EOF or incomplete byte count.

Hardened mode

- The identical packet is received and logged.
- Established TCP state is not deleted.
- Data transfer continues to completion.
- Result is labelled successful mitigation.

The final report will call this an *ICMP-triggered connection abort*, not a TCP RST packet.

Expected Outcome: Throughput Reduction

Legacy vulnerable mode

- Source Quench is accepted for the matching flow.
- Application pacing enters a fixed low-rate state.
- Connection remains open.
- Measured goodput falls and completion takes longer.

Hardened mode

- Source Quench is discarded for transport control.
- Connection remains open.
- Goodput stays near the baseline.
- Current Linux comparison may behave similarly.

Source: RFC 6633

Defense in Depth

Endpoint validation

- verify length and checksum
- match quoted connection
- validate sequence range
- make hard errors nonfatal in synchronized states

Protocol policy

- discard Source Quench
- log suspicious input
- randomize ephemeral ports
- retain legitimate ICMP functions

Network controls

- source-address validation
- stateful ICMP inspection
- rate limiting where appropriate
- avoid blanket ICMP blocking

The strongest practical defense is careful validation and minimum nonfatal reaction, because legitimate ICMP errors may originate from intermediate routers.

Source: RFC 5927; RFC 6056; RFC 3704

Why the Design Should Work

1. The setup fixes every laboratory address and TCP port, so the quoted flow identifier is deterministic.
2. Both attacks reuse the same IPv4, ICMP, checksum and quoted-TCP construction code.
3. The controlled handler makes the historical reaction observable without an obsolete operating system.
4. The same packet is applied to legacy and hardened policies, isolating the defensive decision.
5. Packet capture, endpoint state and goodput provide independent evidence of the outcome.



Limitations and Assumptions

Assumptions

- owned, isolated Linux-based environment
- fixed private addresses and ports
- no live sniffing by Host A
- current packet reaches the intended victim
- compatible Python raw-socket permissions

Limitations

- legacy behavior is user-space emulation
- not evidence that modern Linux is vulnerable
- application pacing is not the kernel congestion window
- namespace and WSL2 networking may differ from physical networks
- no Internet-scale tuple discovery or brute force

Conclusion

- The proposal covers both assigned ICMP attacks against TCP in one compact experiment.
- Three Linux network namespaces replace multiple virtual machines and keep setup time low.
- A group-written packet builder produces the exact ICMP error and quoted TCP identifiers.
- Legacy mode provides a repeatable demonstration; hardened mode shows the defense using identical packets.
- The result is evaluated through packet capture, endpoint state and measured goodput.

One packet generator — two historical effects — one defense comparison

References

1. J. Postel, *Internet Control Message Protocol*, RFC 792, September 1981.
2. R. Braden (Ed.), *Requirements for Internet Hosts — Communication Layers*, RFC 1122, October 1989, Section 4.2.3.9.
3. F. Gont, *ICMP Attacks against TCP*, RFC 5927, July 2010.
4. F. Gont, *Deprecation of ICMP Source Quench Messages*, RFC 6633, May 2012.
5. M. Larsen and F. Gont, *Recommendations for Transport Protocol Port Randomization*, RFC 6056, January 2011.
6. F. Baker and P. Savola, *Ingress Filtering for Multihomed Networks*, RFC 3704 and BCP 84, March 2004.

Thank You